

排序问题-笔记

```
vector<pair<int,int>> a(n);
for(int i=0;i<n;i++) cin>>a[i].first>>a[i].second;
```

C++

```
sort(a.begin(),a.end(),[](const pair<int,int>& a,const pair<int,int>& b) {
    return a.second < b.second;
});
```

C++

```
struct Student {
    std::string name;
    int score;
    int classId;
    Student(std::string n, int s, int c) : name(n), score(s), classId(c) {}
};
```

C++

```
int main() {
    std::vector<Student> students = {
        {"Alice", 85, 1},
        {"Bob", 92, 2},
        {"Charlie", 85, 1},
        {"David", 78, 3},
        {"Eve", 92, 2}
    };
}
```

__多关键字排序：先按分数降序，再按班级升序，最后按姓名升序__

```
std::sort(students.begin(), students.end(),[](const Student& a, const Student& b) {
    if (a.score != b.score) return a.score > b.score;
    if (a.classId != b.classId) return a.classId < b.classId;
    return a.name < b.name;
});
```

排序问题-键值对：

```

#include<bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin>>n;
    //用vector<pair<int,int>> (可以保持插入顺序, 允许重复)
    //而不是map<int,int> (自动按键排序, 键不能重复) (如果要按值排序 需要先转换成vector)
    vector<pair<int,int>> a(n);
    for(int i=0;i<n;i++){
        cin>>a[i].first>>a[i].second;
    }

    //经典sort函数写法
    sort(a.begin(),a.end(),
    [](const pair<int,int>& a,const pair<int,int>& b) {
        return a.second < b.second;
    });

    int cnt=0;
    int last_end=-1;

    for(int i=0;i<n;i++){
        // 如果当前活动的开始时间 >= 上一个选择活动的结束时间
        if(a[i].first >= last_end){
            cnt++;
            last_end=a[i].second;
        }
    }
    cout<<cnt<<endl;
    return 0;
}

```

字符串去重后排序，我们需要对字符串进行两个操作：

1. **去重**：去掉重复出现的字符
2. **排序**：按照字符的ASCII码从小到大排序

思路

方法1：使用集合（Set）

- 1.将字符串所有字符放入set《char》中（自动去重和排序）
- 2.将set中的字符拼接成新字符串输出

方法2：使用排序和去重算法

1. 先对字符串排序（使用sort）
2. 再用unique算法去重
3. 截取有效部分输出

方法3：使用标记数组

1. 创建一个大小为128（ASCII码范围）的bool数组
2. 遍历字符串，标记出现过的字符
3. 按ASCII码顺序输出被标记的字符

C++代码实现

方法1：使用Set（最简洁）

```
C++  
  
#include<bits/stdc++.h>  
using namespace std;  
int main() {  
    string s;  
    getline(cin, s);  
    set<char> charSet;  
    // _ 将字符插入set (自动去重和排序) _  
    for(char c : s) {  
        charSet.insert(c);  
    }  
    // _ 输出结果_  
  
    for(char c : charSet)    cout << c;  
  
    cout << endl;  
    return 0;  
}
```

方法2：使用排序和去重算法

```
C++  
  
#include <iostream>  
#include <algorithm>  
#include <string>  
using namespace std;  
int main() {  
    string s;  
    getline(cin, s);  
    // 1._ 排序_  
    sort(s.begin(), s.end());  
    // 2._ 去重 (unique将重复元素移到末尾, 返回新的结尾迭代器) _  
    auto new_end = unique(s.begin(), s.end());  
    // 3._ 截取去重后的部分并输出_  
    s.erase(new_end, s.end());  
    cout << s << endl;  
    return 0;  
}
```

方法3：使用标记数组（效率最高）

```

#include <iostream>
#include <string>
using namespace std;
int main() {
    string s;
    getline(cin, s);
    // ASCII__码有128个, 创建标记数组_
    bool exists[128] = {false};  _//_ _初始化为false_
    //_ _标记出现过的字符_
    for(char c : s) {
        exists[(int)c] = true;
    }

    //_ _按ASCII码顺序输出被标记的字符_

    for(int i = 0; i < 128; i++) {
        if(exists[i])
            cout << (char)i;
    }
    cout << endl;
    return 0;
}

```

代码详解

方法1 (Set) 的优点:

- 代码简洁明了
- Set自动维护排序和唯一性
- 适合初学者理解

方法2 (排序+去重) 的优点:

- 使用标准库算法, C++风格
- 不需要额外数据结构
- 原地操作, 节省空间

方法3 (标记数组) 的优点:

- 时间复杂度 $O(n)$, 效率最高
- 适合字符集有限的情况 (如ASCII)
- 内存占用小 (128个bool)

示例验证

输入:

text

ad2f3adjfeainzzzv

处理过程：

1. 原始字符：a d 2 f 3 a d j f e a i n z z z v
2. 去重后：a d 2 f 3 j e i n z v
3. 排序后：2 3 a d e f i j n v z

输出：

text

23adefijnvz

推荐使用

对于这道题，推荐使用方法3（标记数组），因为：

1. ASCII码只有128个，数组大小固定
2. 时间复杂度 $O(n)$ ，效率最高
3. 代码简单，容易理解

如果字符范围不确定（如Unicode），则使用方法1（Set）更合适。